CSCE 46103/56103 Artificial Intelligence – Fall 2026

Neural Networks: Architectures and Evolution

Jungtaek Kim
jungtaek.kim@uark.edu

University of Arkansas

November 5, 2026

This Material's Topics

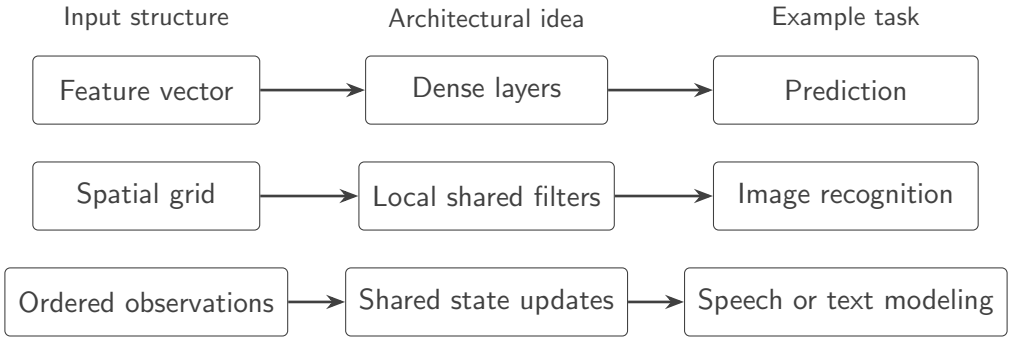
- Learned representations: neurons, hidden layers, memory, and reconstruction.
- Structure in images and sequences: convolution, recurrence, gates, and shared weights.
- Deeper and combined models: residual paths, speech recognition, language modeling, and image captioning.

The Central Question

How did changes in neural-network architecture make new kinds of learning possible?

Source: Russell and Norvig [2020], Goodfellow et al. [2016].

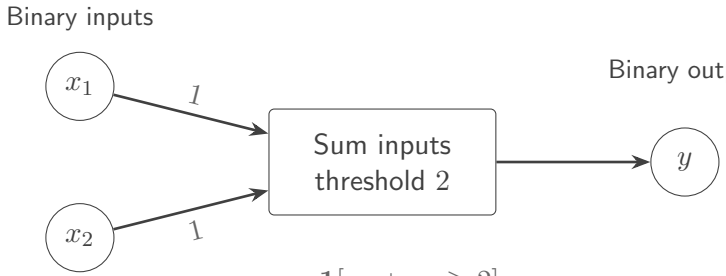
Architectural Families and Modalities



One architecture can serve several modalities; components can be combined.

Source: Goodfellow et al. [2016].

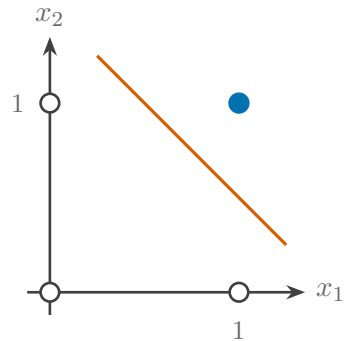
McCulloch–Pitts Neurons



1943: a logical computation built from idealized threshold units.

Adapted from: McCulloch and Pitts [1943]; excitatory-only AND example. Connections are specified, not learned.

Rosenblatt's Perceptron



$$s = \mathbf{w}^\mathsf{T}\mathbf{x} + b$$
$$\hat{y} = \mathbf{1}[s \geq 0]$$

Example: $\mathbf{w} = (1, 1)^\mathsf{T}$, $b = -1.5$
orange line: $x_1 + x_2 = 1.5$

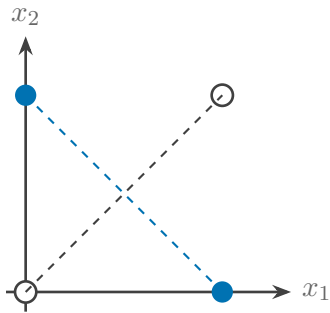
○ class 0 ● class 1

Learning changes $\mathbf{w}$ and b, moving the boundary.

1958: learning adjusts a decision rule from labeled examples.

Source: Rosenblatt [1958]. Linear-threshold abstraction; displayed weights are an illustrative example.

From a Single Layer to Hidden Units



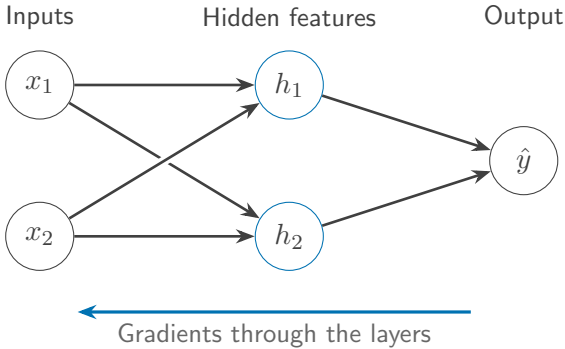
A single straight boundary cannot put both blue points on one side and both white points on the other.

class 0 class 1

One linear boundary cannot separate XOR; hidden units can change the representation.

Source: Minsky and Papert [1969], Goodfellow et al. [2016]. This limitation concerns a single linear-threshold classifier.

Multilayer Perceptrons

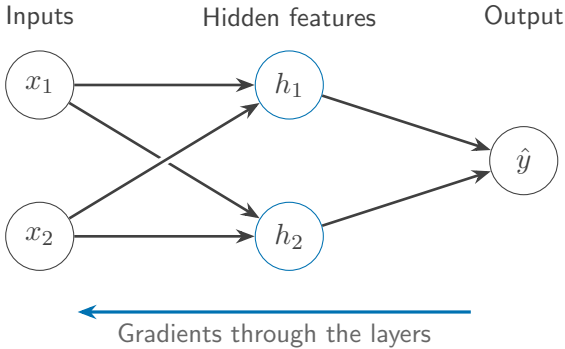


$$h_1 = \mathrm{ReLU}(x_1 - x_2)$$
$$h_2 = \mathrm{ReLU}(x_2 - x_1)$$
$$\hat{y} = h_1 + h_2$$

On binary inputs,
$\hat{y} = \mathrm{XOR}(x_1, x_2)$.

Source: Rumelhart et al. [1986], Goodfellow et al. [2016]. The 1986 paper helped popularize backpropagation; the XOR construction is illustrative.

Multilayer Perceptrons



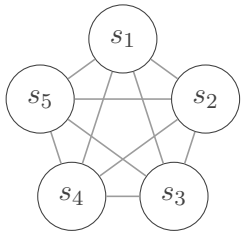
$$h_1 = \mathrm{ReLU}(x_1 - x_2)$$
$$h_2 = \mathrm{ReLU}(x_2 - x_1)$$
$$\hat{y} = h_1 + h_2$$

On binary inputs,
$\hat{y} = \mathrm{XOR}(x_1, x_2)$.

Nonlinear hidden features increase expressiveness; backpropagation computes training gradients.

Source: Rumelhart et al. [1986], Goodfellow et al. [2016]. The 1986 paper helped popularize backpropagation; the XOR construction is illustrative.

Hopfield Networks



Symmetric connections
no self-connections

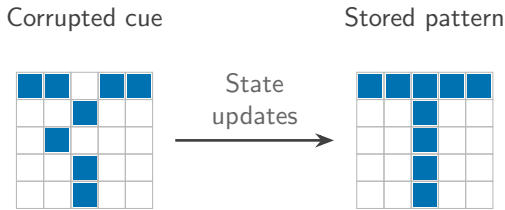
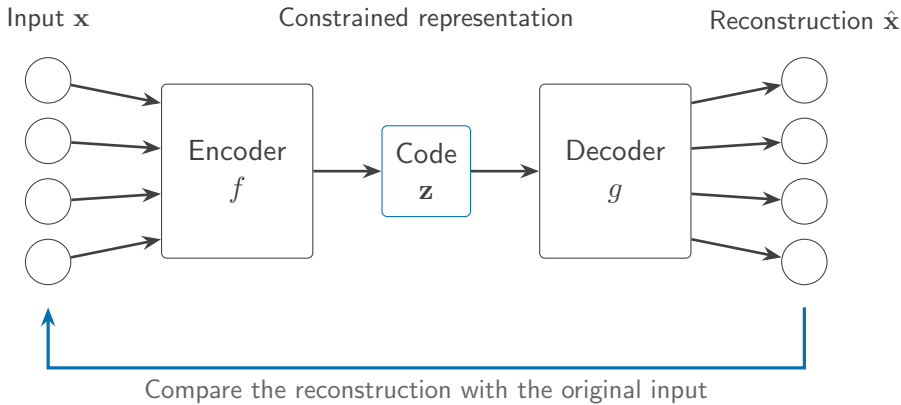


Illustration of content-addressable retrieval

1982: recurrent interactions support retrieval from a partial or corrupted cue.

Adapted from: Hopfield [1982]. Binary pattern illustration; retrieval depends on stored patterns and the initial state.

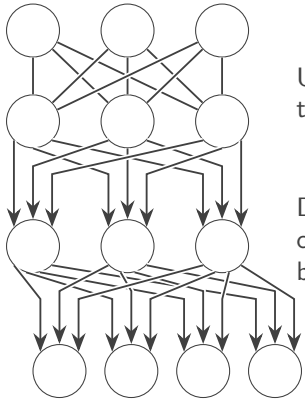
Autoencoders



Learn a representation by reconstructing the input through a constrained code.

Source: Goodfellow et al. [2016, Ch. 14]; deep autoencoders: Hinton and Salakhutdinov [2006].

Deep Belief Networks and Pretraining



Learn one layer at a time

↓

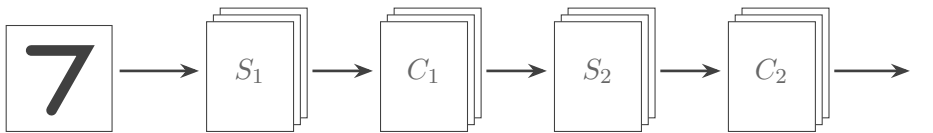
Initialize a deep model

2006: layerwise unsupervised learning helped initialize deep models.

Adapted from: Hinton et al. [2006]. A deep belief network has an undirected top layer pair and directed generative connections below.

The Neocognitron

1980 Hierarchical visual recognition



S-cells: local feature selectivity C-cells: tolerance to position shifts

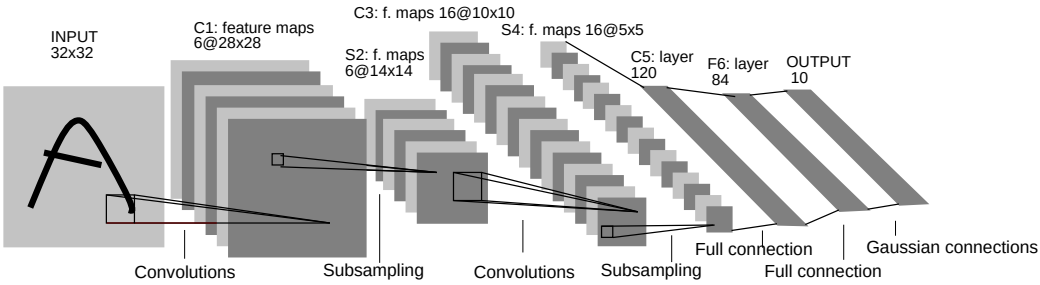
Original learning was self-organizing, not end-to-end backpropagation.

Alternate local feature detection and tolerance to small position changes.

Adapted from: Fukushima [1980]; simplified architecture schematic.

LeNet-5

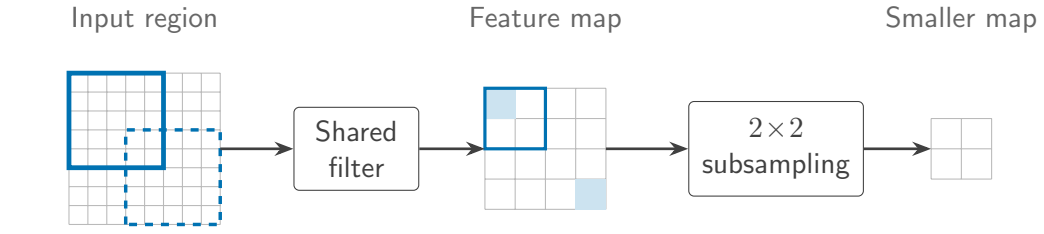
1998 Gradient-based document recognition



A complete trainable system for recognizing handwritten characters.

Figure from: LeCun et al. [1998, Fig. 2]. Original output units use radial basis functions.

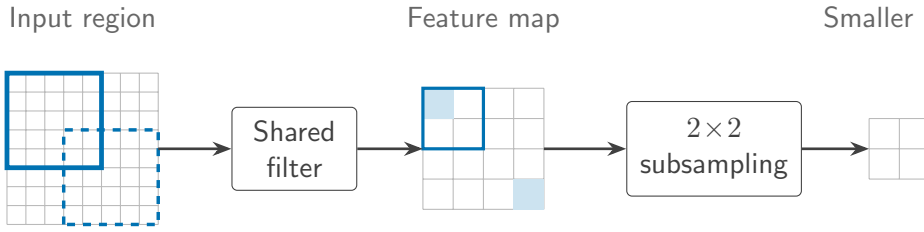
LeNet-5: Locality and Shared Weights



Illustrative spatial sizes; original LeNet-5 uses learned subsampling.

Adapted from: LeCun et al. [1998]; simplified architecture schematic.

LeNet-5: Locality and Shared Weights



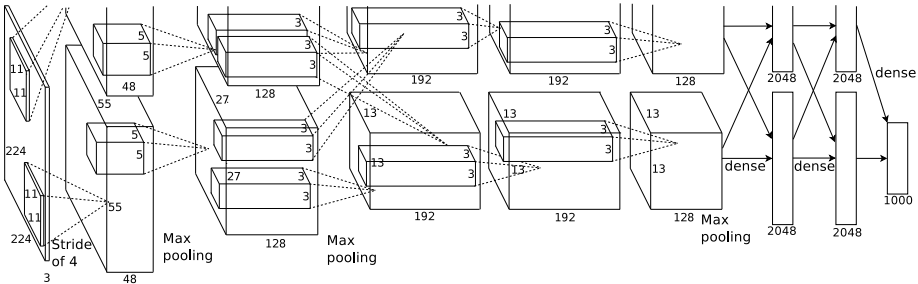
Illustrative spatial sizes; original LeNet-5 uses learned subsampling.

One detector is reused across positions; subsampling reduces spatial resolution.

Adapted from: LeCun et al. [1998]; simplified architecture schematic.

AlexNet

2012 ImageNet classification

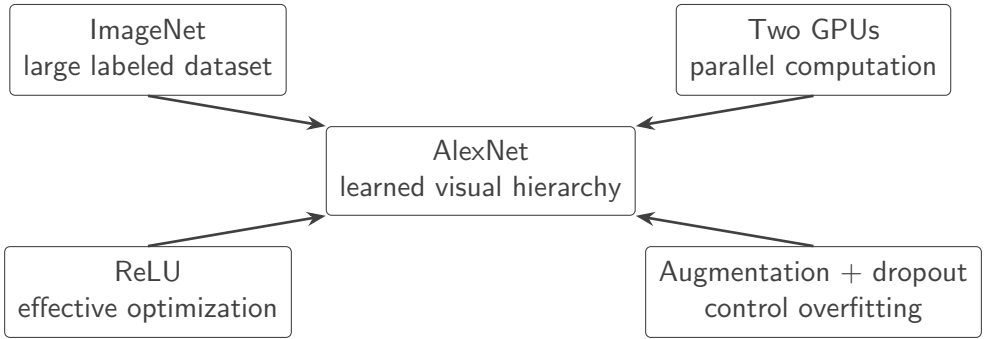


Five convolutional layers and three fully connected layers at ImageNet scale.

Figure from: Krizhevsky et al. [2012, Fig. 2]. The two rows show the original split across two GPUs.

AlexNet: Architecture and Scale

A successful combination of existing and developing ideas

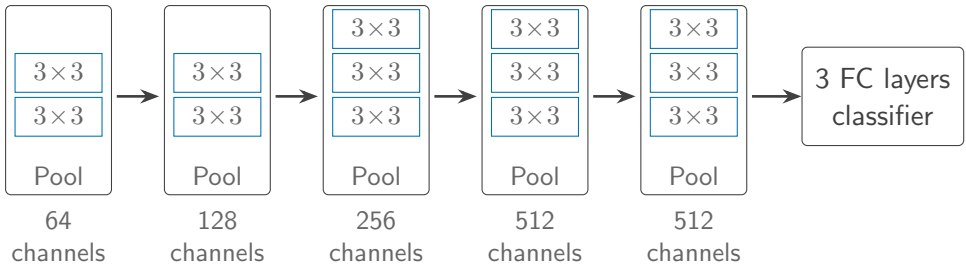


The advance combined a network with data, computation, and effective training.

Source: Krizhevsky et al. [2012].

VGG Networks

VGG-16 introduced 2014; published 2015

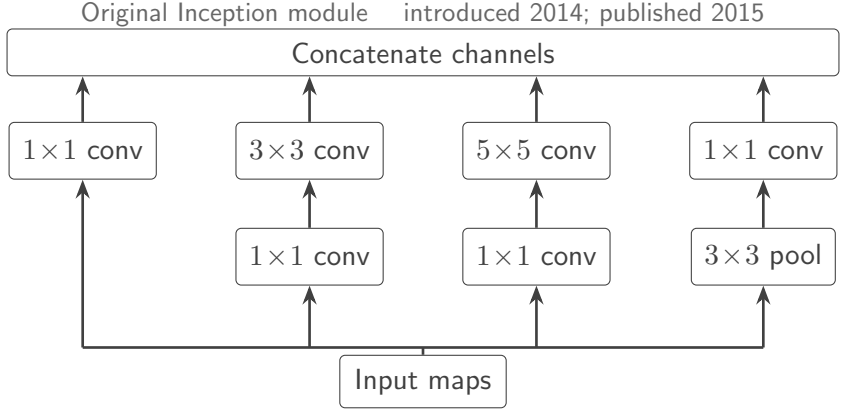


13 convolutional layers + 3 fully connected layers = 16 learned layers.

Build depth by repeating small convolutional filters.

Adapted from: Simonyan and Zisserman [2015]; simplified architecture schematic.

GoogLeNet and Inception



Parallel branches capture different scales; 1×1 layers control channel counts.

Adapted from: Szegedy et al. [2015]; simplified architecture schematic.

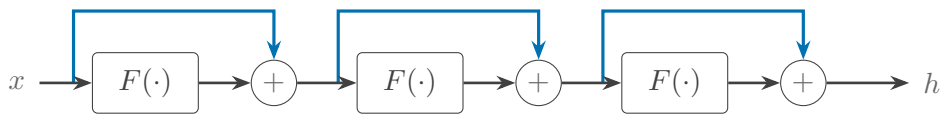
Residual Networks

ResNet introduced 2015; published 2016

Plain network



Residual network



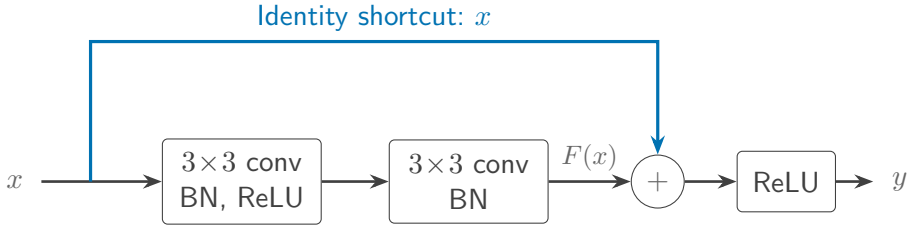
Each shortcut carries its block input forward; nonlinearities are omitted in this overview.

Learn a change to the representation while retaining an identity path.

Adapted from: He et al. [2016]; simplified architecture schematic.

ResNet: The Residual Block

Original basic block, with matching input and output dimensions



$$u = x + F(x), \qquad y = \mathrm{ReLU}(u)$$

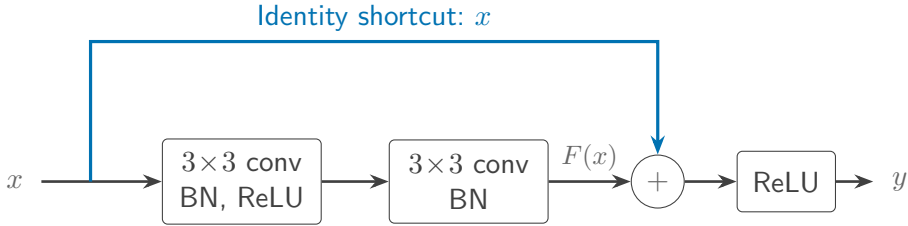
At the addition: $\dfrac{\partial u}{\partial x} = I + \dfrac{\partial F}{\partial x}$

BN: batch normalization. Dimension changes require an adjusted shortcut.

Adapted from: He et al. [2016]; simplified architecture schematic.

ResNet: The Residual Block

Original basic block, with matching input and output dimensions



$$u = x + F(x), \qquad y = \mathrm{ReLU}(u)$$

At the addition: $\dfrac{\partial u}{\partial x} = I + \dfrac{\partial F}{\partial x}$

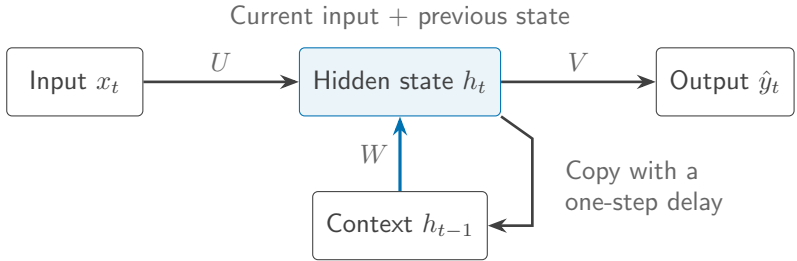
BN: batch normalization. Dimension changes require an adjusted shortcut.

The addition supplies a direct path for both information and gradients.

Adapted from: He et al. [2016]; simplified architecture schematic.

Elman Networks (1990)

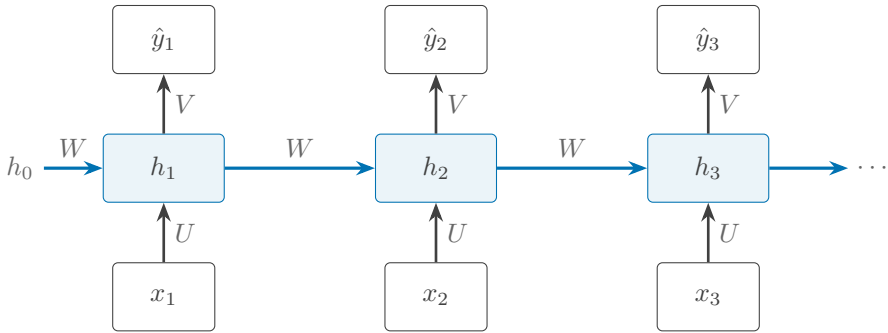
$$h_t = \sigma(U x_t + W h_{t-1} + b)$$



A delayed copy of the hidden state supplies context for the next input.

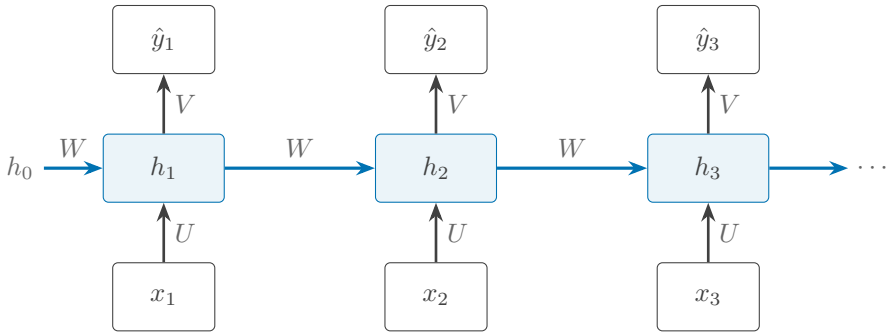
Adapted from: Elman [1990]; simplified architecture schematic.

An Elman Network Across Time



Adapted from: Elman [1990], Goodfellow et al. [2016]; simplified architecture schematic.

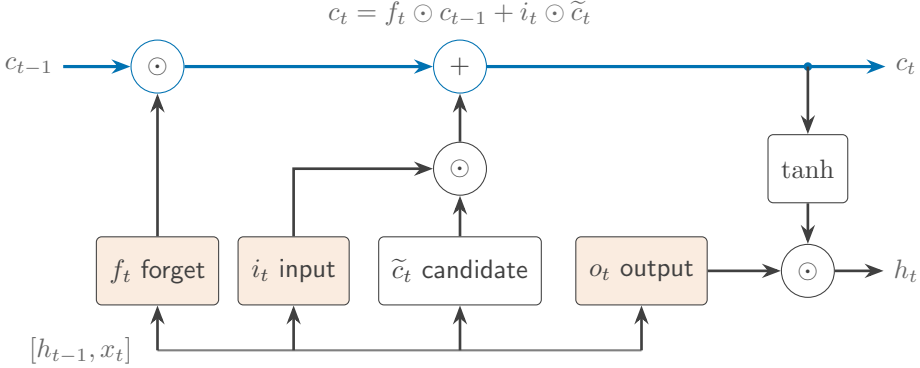
An Elman Network Across Time



The same network and weights are reused at every step.

Adapted from: Elman [1990], Goodfellow et al. [2016]; simplified architecture schematic.

Long Short-Term Memory



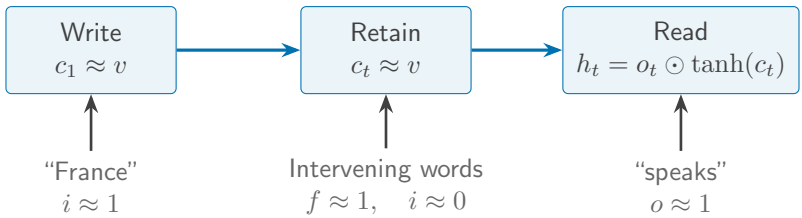
Gates: sigmoid outputs in $(0, 1)$; $\odot$: elementwise multiplication.

A gated memory path separates retaining information from exposing an output.

Adapted from: Hochreiter and Schmidhuber [1997]; forget-gate extension: Gers et al. [2000].

An LSTM Memory Path

"She grew up in France ... she speaks French."



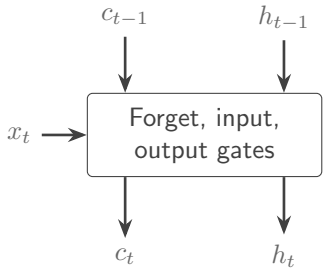
Illustrative gates and one memory component, starting from $c_0 = 0$.

Learned gates can preserve useful information across intervening inputs.

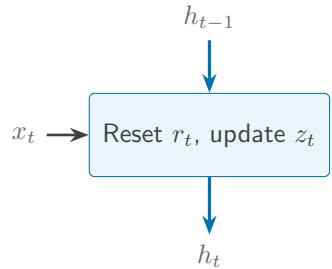
Source: Hochreiter and Schmidhuber [1997], Gers et al. [2000].

Gated Recurrent Units (2014)

LSTM: cell state + hidden state



GRU: hidden state

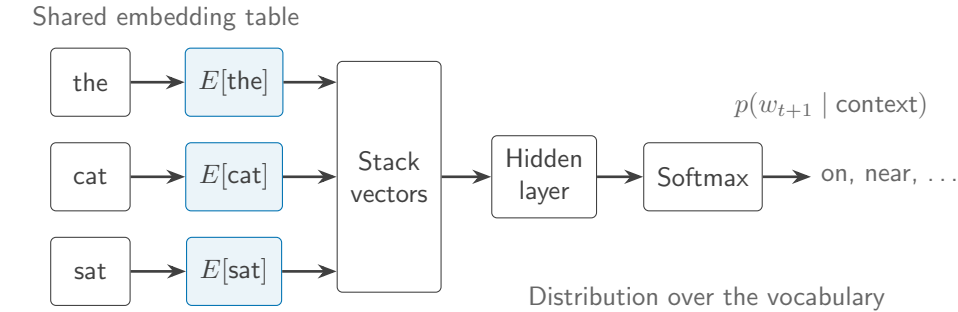


$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \widetilde{h}_t$$

A GRU uses one state vector; here, a larger update gate retains more old state.

Adapted from: Cho et al. [2014], Goodfellow et al. [2016]; simplified architecture schematic.

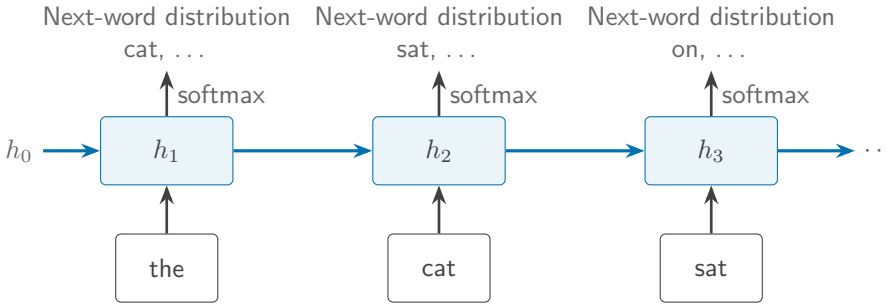
A Neural Probabilistic Language Model (2003)



Learn word vectors and a next-word predictor together, using a fixed context window.

Adapted from: Bengio et al. [2003, Fig. 1]; simplified architecture.

A Recurrent Language Model (2010)

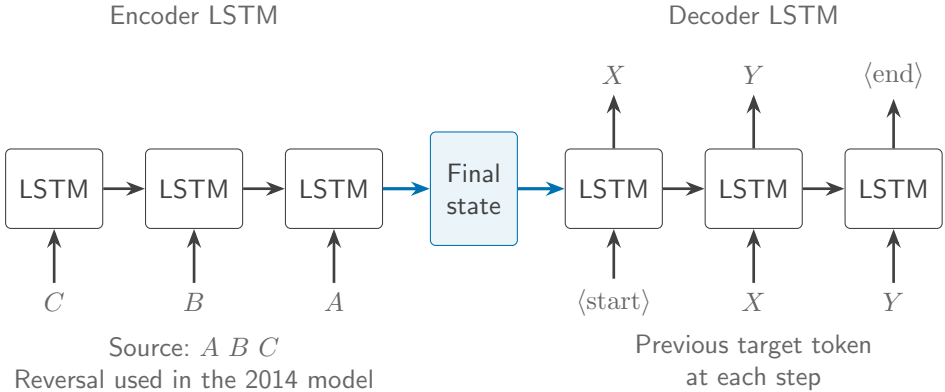


Illustrative word sequence; hidden states summarize the preceding context.

A recurrent state carries context beyond a fixed window of preceding words.

Adapted from: Mikolov et al. [2010, Fig. 1]; unfolded teaching schematic.

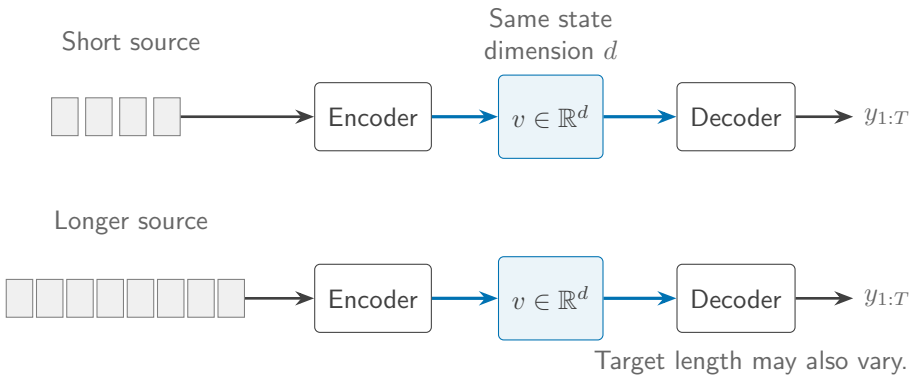
LSTM Encoder–Decoder Networks (2014)



An encoder and a decoder connect variable-length sequences through a fixed-size state.

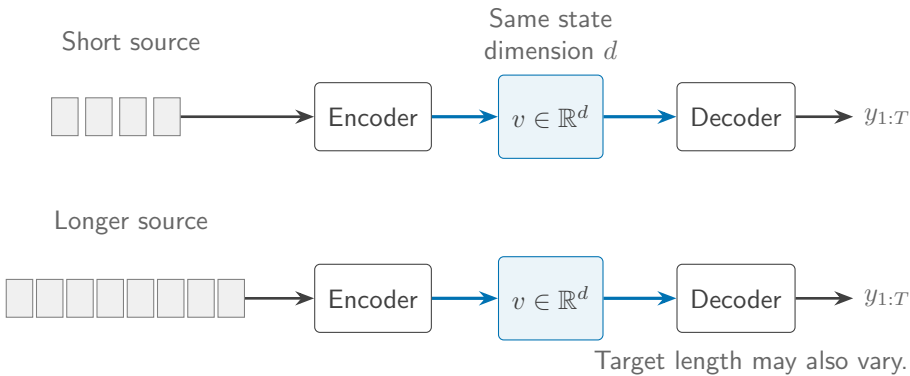
Adapted from: Sutskever et al. [2014, Fig. 1]; each block summarizes a multilayer LSTM.

The Encoder–Decoder Bottleneck



Adapted from: Cho et al. [2014], Sutskever et al. [2014]; simplified architecture schematic.

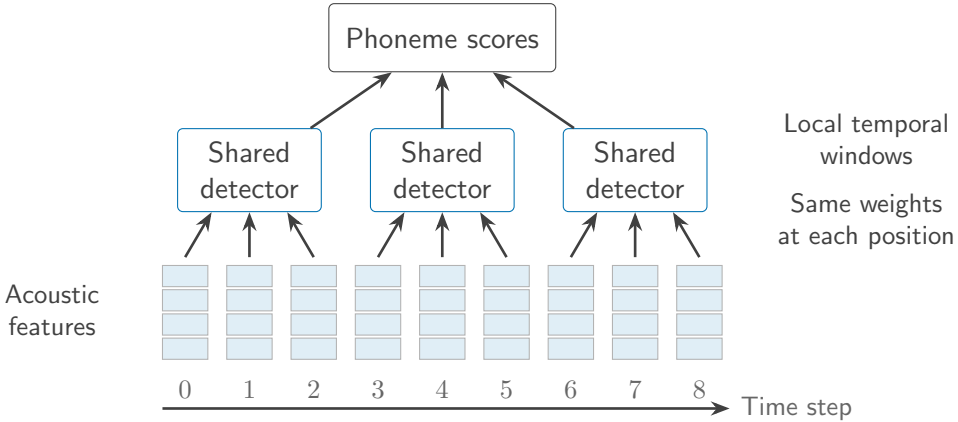
The Encoder–Decoder Bottleneck



All source information reaches this decoder through the encoder's final state.

Adapted from: Cho et al. [2014], Sutskever et al. [2014]; simplified architecture schematic.

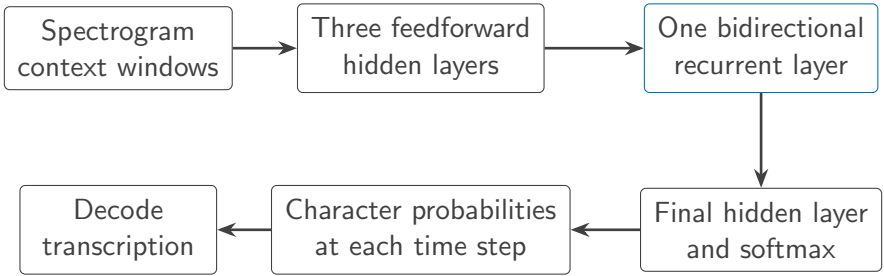
Time-Delay Neural Networks



1989: reuse acoustic feature detectors across positions in time.

Adapted from: Waibel et al. [1989]; schematic temporal windows, not the original layer dimensions.

Deep Speech

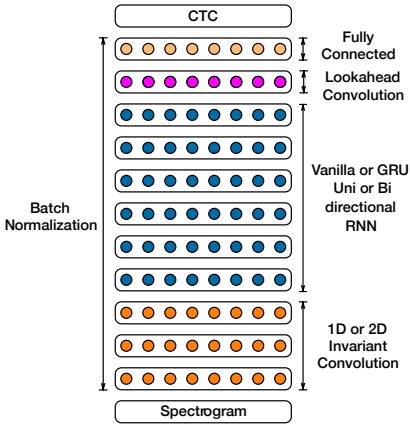


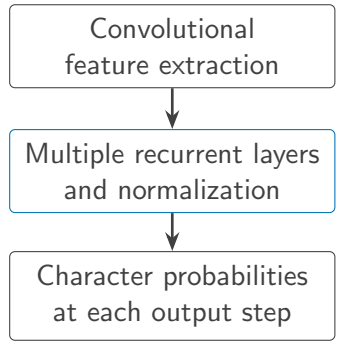
The recurrent layer reads acoustic context in both temporal directions.

2014: spectrograms to character probabilities, trained from audio–transcript pairs.

Adapted from: Hannun et al. [2014, Sec. 2]. Decoding also uses a language model.

Deep Speech 2

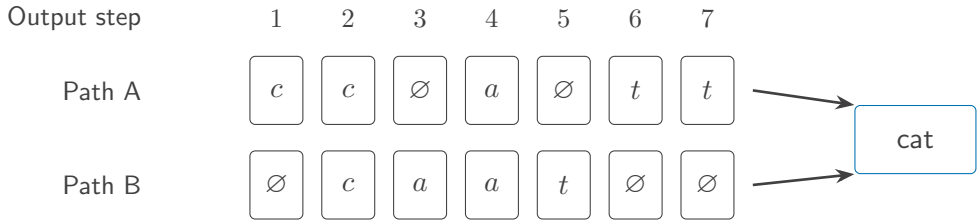




Combine convolutional inputs with deeper recurrent processing.

Figure from: Amodei et al. [2016, Fig. 1]; architecture options shown in the original. Introduced in 2015; published in 2016.

Deep Speech 2: Learning from Transcripts

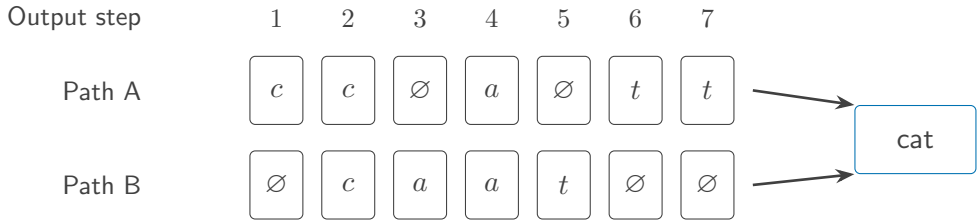


Merge adjacent repeats, then remove blanks (∅).

Many frame-level paths can correspond to the same transcript.

Source: Graves et al. [2006], Amodei et al. [2016]. Two illustrative alignments; CTC is the training criterion, not a network architecture.

Deep Speech 2: Learning from Transcripts



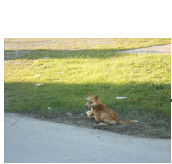
Merge adjacent repeats, then remove blanks (∅).

Many frame-level paths can correspond to the same transcript.

CTC sums over valid alignments; training needs the transcript.

Source: Graves et al. [2006], Amodei et al. [2016]. Two illustrative alignments; CTC is the training criterion, not a network architecture.

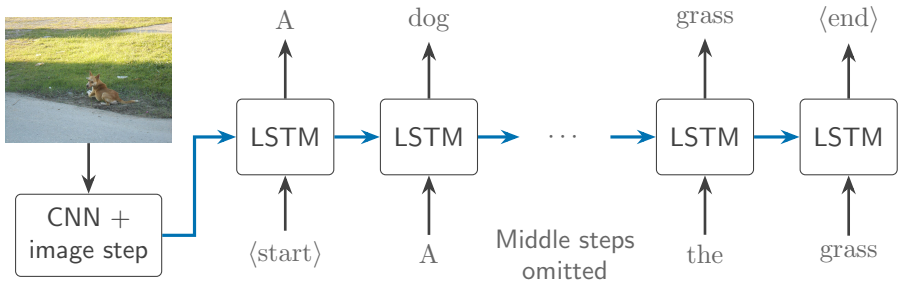
Show and Tell (2015)



A CNN encodes the image; an LSTM generates the description one word at a time.

Adapted from: Vinyals et al. [2015, Fig. 3]; simplified information flow.
Image: Editor abcdef, *Dog lying on grass* (2015), CC0. Caption is illustrative.

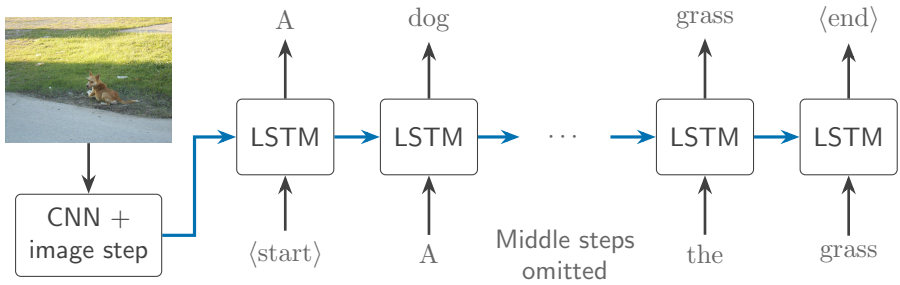
Show and Tell: Caption Generation



Illustrative caption: "A dog lies on the grass."

Source: Vinyals et al. [2015].
Image: Editor abcdef, *Dog lying on grass* (2015), CC0. Caption is illustrative.

Show and Tell: Caption Generation

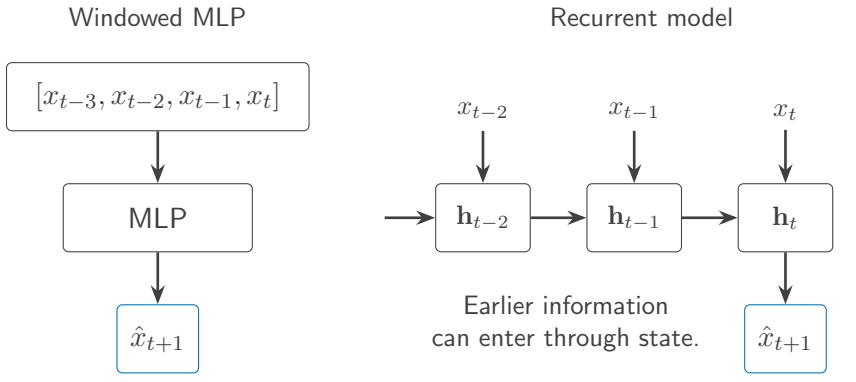


Illustrative caption: "A dog lies on the grass."

Each selected word becomes the next input; the end token finishes the caption.

Source: Vinyals et al. [2015].
Image: Editor abcdef, *Dog lying on grass* (2015), CC0. Caption is illustrative.

MLPs and Recurrent Networks for Forecasting

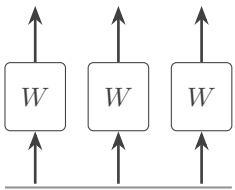


The same observations can enter a fixed window or a shared recurrent update.

Source: Goodfellow et al. [2016, Chs. 6 and 10]. Illustrative forecasting designs; both use only observations available through time t.

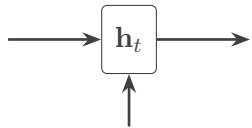
Comparing Architectural Changes

Local sharing



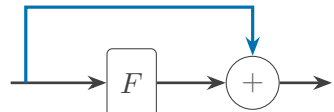
Reuse a detector across positions

Recurrent state



Carry information between steps

Residual path



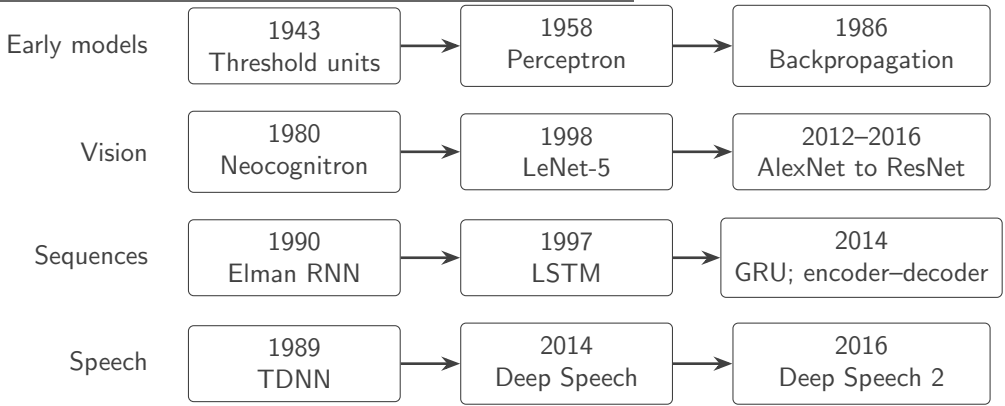
Add an identity route alongside a learned change

CNNs can process audio; recurrent models can process words or measurements.

These are reusable design ideas, not mutually exclusive model categories.

Source: Goodfellow et al. [2016], He et al. [2016].

Historical Development Revisited



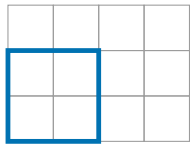
2015: Show and Tell combines convolutional vision and recurrent language.

Several lines of development coexist and combine.

Sources: Goodfellow et al. [2016], He et al. [2016], Amodei et al. [2016], Vinyals et al. [2015]. Selected publication milestones; horizontal positions are not a uniform time scale.

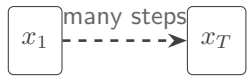
Checkpoint: Explaining a Design Choice

Case 1



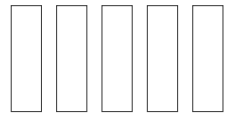
The same visual feature appears in many positions.

Case 2



An early observation matters much later.

Case 3



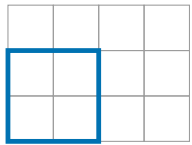
A much deeper network is difficult to optimize.

Which architectural change addresses each problem, and why?

Source: LeCun et al. [1998], Hochreiter and Schmidhuber [1997], Cho et al. [2014], He et al. [2016].

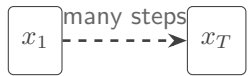
Checkpoint: Explaining a Design Choice

Case 1



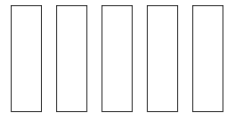
The same visual feature appears in many positions.

Case 2



An early observation matters much later.

Case 3



A much deeper network is difficult to optimize.

Which architectural change addresses each problem, and why?

Case 1	**Case 2**	**Case 3**
LeNet / CNN	LSTM / GRU	ResNet
Shared local filters	Gated memory	Residual paths

Source: LeCun et al. [1998], Hochreiter and Schmidhuber [1997], Cho et al. [2014], He et al. [2016].

UNIVERSITY OF ARKANSAS

Main Takeaways

- Hidden layers learn useful features and representations.
- Locality, sequence order, and shared computation reflect structure in the data.
- Memory, gates, shortcuts, and encoder–decoder composition guide information flow.
- Architecture, training methods, data, and computation developed together.

Source: Goodfellow et al. [2016].

Thank You

Any questions?

References

D. Amodei, S. Ananthanarayanan, R. Anubhai, J. Bai, E. Battenberg, C. Case, J. Casper, B. Catanzaro, Q. Cheng, G. Chen, J. Chen, J. Chen, Z. Chen, M. Chrzanowski, A. Coates, G. Diamos, K. Ding, N. Du, E. Elsen, J. Engel, W. Fang, L. Fan, C. Fougner, L. Gao, C. Gong, A. Hannun, T. Han, L. Johannes, B. Jiang, C. Ju, B. Jun, P. LeGresley, L. Lin, J. Liu, Y. Liu, W. Li, X. Li, D. Ma, S. Narang, A. Ng, S. Ozair, Y. Peng, R. Prenger, S. Qian, Z. Quan, J. Raiman, V. Rao, S. Satheesh, D. Seetapun, S. Sengupta, K. Srinet, A. Sriram, H. Tang, L. Tang, C. Wang, J. Wang, K. Wang, Y. Wang, Z. Wang, Z. Wang, S. Wu, L. Wei, B. Xiao, W. Xie, Y. Xie, D. Yogatama, B. Yuan, J. Zhan, and Z. Zhu. Deep Speech 2: End-to-end speech recognition in English and Mandarin. In *Proceedings of the International Conference on Machine Learning*, 2016. URL `https://proceedings.mlr.press/v48/amodei16.html`.

Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003. URL `https://jmlr.org/papers/v3/bengio03a.html`.

References

K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2014. doi: 10.3115/v1/D14-1179. URL https://aclanthology.org/D14-1179/.

J. L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990. doi: 10.1207/s15516709cog1402_1.

K. Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36(4):193–202, 1980. doi: 10.1007/BF00344251.

F. A. Gers, J. Schmidhuber, and F. Cummins. Learning to forget: Continual prediction with LSTM. *Neural Computation*, 12(10):2451–2471, 2000. doi: 10.1162/089976600300015015.

I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016. URL https://www.deeplearningbook.org/.

References

A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the International Conference on Machine Learning*, 2006. doi: 10.1145/1143844.1143891. URL https://www.cs.toronto.edu/~graves/icml_2006.pdf.

A. Hannun, C. Case, J. Casper, B. Catanzaro, G. Diamos, E. Elsen, R. Prenger, S. Satheesh, S. Sengupta, A. Coates, and A. Y. Ng. Deep Speech: Scaling up end-to-end speech recognition. *arXiv preprint arXiv:1412.5567*, 2014. URL https://arxiv.org/abs/1412.5567.

K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016. URL https://www.cv-foundation.org/openaccess/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html.

G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006. doi: 10.1126/science.1127647.

G. E. Hinton, S. Osindero, and Y.-W. Teh. A fast learning algorithm for deep belief nets. *Neural Computation*, 18(7):1527–1554, 2006. doi: 10.1162/neco.2006.18.7.1527.

References

S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, 1982. doi: 10.1073/pnas.79.8.2554.

A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, 2012. URL `https://papers.nips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html`.

Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791. URL `https://leon.bottou.org/papers/lecun-98h`.

W. S. McCulloch and W. Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5:115–133, 1943. doi: 10.1007/BF02478259.

References

T. Mikolov, M. Karafiát, L. Burget, J. Černocký, and S. Khudanpur. Recurrent neural network based language model. In *Proceedings of the Annual Conference of the International Speech Communication Association*, 2010. doi: 10.21437/Interspeech.2010-343. URL https://www.isca-archive.org/interspeech_2010/mikolov10_interspeech.html.

M. L. Minsky and S. A. Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969. URL https://mitpress.mit.edu/9780262130431/perceptrons/.

F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. doi: 10.1037/h0042519.

D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986. doi: 10.1038/323533a0.

S. J. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020. URL https://aima.cs.berkeley.edu/.

References

K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *Proceedings of the International Conference on Learning Representations*, 2015. URL https://www.robots.ox.ac.uk/~vgg/publications/2015/Simonyan15/.

I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, 2014. URL https://arxiv.org/abs/1409.3215.

C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015. URL https://www.cv-foundation.org/openaccess/content_cvpr_2015/html/Szegedy_Going_Deeper_With_2015_CVPR_paper.html.

O. Vinyals, A. Toshev, S. Bengio, and D. Erhan. Show and Tell: A neural image caption generator. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015. URL https://www.cv-foundation.org/openaccess/content_cvpr_2015/html/Vinyals_Show_and_Tell_2015_CVPR_paper.html.

References

A. Waibel, T. Hanazawa, G. Hinton, K. Shikano, and K. J. Lang. Phoneme recognition using time-delay neural networks. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 37(3):328–339, 1989. doi: 10.1109/29.21701. URL https://www.cs.toronto.edu/~hinton/absps/waibelTDNN.pdf.